



FOR SOFTWARE ENGINEERS

Kafka Down. Checkout Succeeds.

GraphQL Federation and Kafka, with Spring Boot and Micronaut

One checkout, two frameworks, one supergraph, and every failure measured.

A checkout split the way real teams split it: orders in Spring Boot, payments in Micronaut, composed by the Apollo Router, with Kafka between them. Everything on this sheet was built and measured, not assumed.

Spring Boot 4.1.1 and Micronaut 5.1.5 on Java 25, Apollo Router 2.17.0, Kafka 4.2.1, Postgres 17.

Every version on this sheet was current on 25 September 2026 and will not stay current. The method is the durable part: run the script, read what came back, and write down the result rather than the expectation.

In scope

- Federation: keys, entities, the query plan, composition, ownership
- N plus one on both sides, and what the router does and does not batch
- HTTP 200, partial data and null propagation
- Kafka: coupling, the outbox, at least once, poison messages
- Annotations and configuration compared, job by job

Out of scope

- Performance benchmarks, which were not measured
- Subscriptions
- Anything about how this channel is produced, which is not the subject

Numbers are from one run of the script. Timings vary slightly between runs; the conclusions do not.

01 The findings, in one table

N plus one, 20 orders	20 and 20 selects naive; 1 and 1 batched
A subgraph down	HTTP 200, payment null, error tagged
payment made non-null	the whole order null
Kafka down, publish in the request	refused after about 60 seconds
Kafka down, outbox	accepted, survived a crash, paid
Crash before the offset commit	2 payments without a key, 1 with
Poison message, defaults	Spring 10 attempts; Micronaut skipped the poll
Order.id ID! against String!	composition refused

Every row is reproduced by `scripts/verify.sh`.

02 One checkout, two teams

Orders is a Spring Boot subgraph, payments a Micronaut subgraph, each with its own Postgres database. The Apollo Router composes them into one GraphQL endpoint. Kafka carries order-placed and payment-captured between them.

Placing an order writes the order, its lines and an outbox row in one transaction. A relay publishes the event, payments captures the money and publishes back, and orders marks the order paid.

```
orders-spring: schema.graphqls

extend schema @link(url: "https://specs.apollo.dev/federation/v2.9", import: ["@key"])

type Query {
  order(id: ID!): Order
  ordersByCustomer(customerId: ID!): [Order!]!
}

type Mutation {
  placeOrder(input: PlaceOrderInput!): Order!
}

type Order @key(fields: "id") {
  id: ID!
  customerId: ID!
  currency: String!
  total: Int!
  status: OrderStatus!
  lines: [OrderLine!]!
}

type OrderLine {
  sku: String!
  quantity: Int!
  unitPrice: Int!
}

enum OrderStatus {
  PLACED
  PAID
}

input PlaceOrderInput {
  customerId: ID!
  currency: String!
  lines: [LineInput!]!
}

input LineInput {
  sku: String!
  quantity: Int!
}
```

payments-micronaut: schema.graphqls

```
extend schema @link(url: "https://specs.apollo.dev/federation/v2.9", import: ["@key"])

type Query {
  payment(id: ID!): Payment
}

type Order @key(fields: "id") {
  id: ID!
  payment: Payment
}

type Payment {
  id: ID!
  amount: Int!
  currency: String!
  status: PaymentStatus!
  order: Order!
}

enum PaymentStatus {
  CAPTURED
}
```

One query returned the total and lines from Spring and the payment from Micronaut; the query plan fetched orders, then payments through `_entities`.

Run on a real machine, 2026-09-25. Reproduce it with `scripts/verify.sh` in the course repository.

03 Spring: annotations do the wiring

Query mapping and mutation mapping bind methods to fields by name. Batch mapping resolves a field for every parent in the response at once. Entity mapping answers the router when it arrives with order IDs from another subgraph: the parameter `idList` has the word `List` stripped to find the key.

A federation schema factory and a source builder customizer make the schema a subgraph. Spring Boot 4.1.1 manages `graphql-java` 25, so this service uses `federation-jvm` 6.2.1, not 7.0.0.



orders-spring: OrderController.java

```
@Controller
public class OrderController {

    private final OrderRepository orders;
    private final OrderLineRepository lines;
    private final OrderService service;

    public OrderController(OrderRepository orders, OrderLineRepository lines, OrderService
service) {
        this.orders = orders;
        this.lines = lines;
        this.service = service;
    }

    @QueryMapping
    public Order order(@Argument Long id) {
        return orders.findById(id).orElse(null);
    }

    @QueryMapping
    public List<Order> ordersByCustomer(@Argument String customerId) {
        return orders.findByCustomerIdOrderById(customerId);
    }

    @MutationMapping
    public Order placeOrder(@Argument PlaceOrderInput input) {
        return service.placeOrder(input);
    }

    // One query for the lines of every order in the response, not one per order.
    @BatchMapping
    public Map<Order, List<OrderLine>> lines(List<Order> batch) {
        Map<Long, List<OrderLine>> byOrder = lines
            .findByOrderIdIn(batch.stream().map(Order::getId).toList())
            .stream()
            .collect(Collectors.groupingBy(OrderLine::getOrderId));
        return batch.stream().collect(Collectors.toMap(
            order → order,
            order → byOrder.getOrDefault(order.getId(), List.of())));
    }

    // The router calls this when another subgraph hands it an Order reference.
    @EntityMapping
    public List<Order> order(@Argument List<Long> idList) {
        Map<Long, Order> found = orders.findAllById(idList).stream()
            .collect(Collectors.toMap(Order::getId, order → order));
        return idList.stream().map(found::get).toList();
    }
}
```

orders-spring: FederationConfig.java

```
@Configuration
public class FederationConfig {

    @Bean
    public FederationSchemaFactory schemaFactory() {
        return new FederationSchemaFactory();
    }

    @Bean
    public GraphQLSourceBuilderCustomizer federation(FederationSchemaFactory factory) {
        return builder → builder.schemaFactory(factory::createGraphQLSchema);
    }
}
```

"If false, indicates to the router that this subgraph doesn't define a reference resolver for this entity." (on @key resolvable)

Apollo federation directive reference, checked 2026-09-25



04 Micronaut: the same job, wired by hand

Micronaut GraphQL has no annotation model. You build the graphql-java engine in a factory: parse the schema, wire each field to a data fetcher, then `Federation.transform` with `fetchEntities` and `resolveEntityType`. Micronaut 5.1.5 manages graphql-java 26, so this service uses `federation-jvm 7.0.0`.

The data loader is registered per request through an execution input customizer. A request scoped `DataLoaderRegistry` bean failed here with "No request present".

payments-micronaut: GraphQLFactory.java

```
@Factory
public class GraphQLFactory {

    @Singleton
    public GraphQL graphQL(ResourceResolver resources, PaymentRepository payments) throws
    Exception {
        TypeDefinitionRegistry types;
        try (var reader = new InputStreamReader(
            resources.getResourceAsStream("classpath:schema.graphqls").orElseThrow(),
            StandardCharsets.UTF_8)) {
            types = new SchemaParser().parse(reader);
        }

        RuntimeWiring wiring = RuntimeWiring.newRuntimeWiring()
            .type("Query", t → t.dataFetcher("payment",
                env → payments.findById(Long.valueOf(env.
<String>getArgument("id"))).orElse(null)))
            .type("Payment", t → t
                .dataFetcher("id", env → env.<Payment>getSource().id().toString())
                .dataFetcher("order", env → new OrderRef(env.
<Payment>getSource().orderId()))
            .type("Order", t → t.dataFetcher("payment", paymentOfOrder()))
            .build();

        GraphQLSchema schema = Federation.transform(types, wiring)
            .fetchEntities(env → env.<List<Map<String,
Object>>>getArgument(_Entity.argumentName)
                .stream()
                .map(representation → new OrderRef((String) representation.get("id")))
                .toList())
            .resolveEntityType(env → env.getSchema().getObjectType("Order"))
            .build();

        return GraphQL.newGraphQL(schema)
            .instrumentation(new MaxQueryDepthInstrumentation(5))
            .build();
    }

    // Every Order in the request asks the same loader, which makes one query.
    static DataFetcher<CompletableFuture<Payment>> paymentOfOrder() {
        return env → env.<String, Payment>getDataLoader(DataLoaders.PAYMENT_BY_ORDER)
            .load(env.<OrderRef>getSource().id());
    }
}
```

payments-micronaut: DataLoaders.java

```
@Singleton
public class DataLoaders implements GraphQLExecutionInputCustomizer {

    static final String PAYMENT_BY_ORDER = "paymentByOrder";

    private final PaymentRepository payments;

    public DataLoaders(PaymentRepository payments) {
        this.payments = payments;
    }

    @Override
    public Publisher<ExecutionInput> customize(ExecutionInput input, HttpRequest request,
                                              @Nullable MutableHttpResponse<String> response)
    {
        DataLoader<String, Payment> byOrder = DataLoaderFactory.newMappedDataLoader(orderIds →
            CompletableFuture.supplyAsync(() → payments.findByIdIn(orderIds).stream()
                .collect(Collectors.toMap(Payment::orderId, Function.identity()))));
        DataLoaderRegistry registry = new DataLoaderRegistry();
        registry.register(PAYMENT_BY_ORDER, byOrder);
        return Publishers.just(input.transform(builder →
            builder.dataLoaderRegistry(registry)));
    }
}
```

A request scoped registry failed with "No request present"; the customizer works.

Run on a real machine, 2026-09-25. Reproduce it with scripts/verify.sh in the course repository.

05 Configuration, side by side

Spring configures GraphQL and Kafka under `spring.graphql` and `spring.kafka`. Micronaut needs `graphql.enabled`, `graphql.path` and `kafka.bootstrap.servers`; offset reset, offset strategy and error strategy sit on the listener annotation in this repository.

```
orders-spring: application.properties
```

```
spring.application.name=orders-spring
server.port=8101
```

```
spring.datasource.url=jdbc:postgresql://localhost:${POSTGRES_PORT:5440}/orders
spring.datasource.username=shop
spring.datasource.password=shop
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
```

```
spring.graphql.http.path=/graphql
```

```
spring.kafka.bootstrap.servers=localhost:9094
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.auto-offset-reset=earliest
```

```
payments-micronaut: application.properties
```

```
micronaut.application.name=payments-micronaut
micronaut.server.port=8102
```

```
graphql.enabled=true
graphql.path=/graphql
```

```
datasources.default.url=jdbc:postgresql://localhost:${POSTGRES_PORT:5440}/payments
datasources.default.username=shop
datasources.default.password=shop
datasources.default.driver-class-name=org.postgresql.Driver
datasources.default.dialect=POSTGRES
datasources.default.schema-generate=CREATE
```

```
kafka.bootstrap.servers=localhost:9094
```

06 Federation batching is not database batching

Twenty orders, one query for each order, its lines and its payment. The router sent payments one `_entities` call holding all twenty orders, both times. The database ran twenty queries per side until the code batched them.

Postgres, one query per order

```
orders: select ... from order_line ol1_0 where ol1_0.order_id=$1
# 20 in all, and 20 for payments
```

Postgres, batched

```
orders: ... where ol1_0.order_id in ($1,$2, ... $20)
payments: ... WHERE (payment_.order_id IN ($1,$2, ... $20))
```

Naive: 20 and 20 selects. Batch mapping and a data loader: 1 and 1.

Run on a real machine, 2026-09-25. Reproduce it with `scripts/verify.sh` in the course repository.

07 HTTP 200, and how far a null travels

With payments stopped, the router answered HTTP 200: the total and status present, payment null, and an error tagged with the service. The operation reached execution; not every field resolved.

Declare payment non-null and run the same outage: the null propagates to the nearest nullable parent, and the whole order comes back null. Architecture decides what can fail; nullability decides how much of the result that failure can erase.

Nullable payment

```
{"data": {"order": {"total": 13900, "status": "PAID", "payment": null}},
"errors": [{"extensions": {"service": "payments", "code": "SUBREQUEST_HTTP_ERROR"}}]}
```

payment: Payment!

```
{"data": {"order": null}, "errors": [...]}
```

Nullable: order survives with payment null. Non-null: order null.

Run on a real machine, 2026-09-25. Reproduce it with `scripts/verify.sh` in the course repository.

08 REST or GraphQL

Neither is the default answer. REST suits simple resource APIs, public APIs that rely on HTTP caching, and file transfer. GraphQL earns its complexity when different clients need different shapes of data from several domains.

The trade-offs

Question	REST	GraphQL
Response shape	server, per resource	client, per query
One screen	a call per resource, or a BFF	one call across services
Caching	standard HTTP caching	deliberate: persisted queries, response cache
Errors	status code	execution errors in the body, often 200
Workload	bounded per endpoint	client controlled: needs cost limits
Evolution	versioned URLs	add and deprecate; removal still breaks

09 Composition, ownership and the directives

Composition refused a supergraph where payments declared `Order.id` as `String!` and orders as `ID!`. It proves the subgraphs agree with each other, not that deployed clients still work.

Each field belongs to the subgraph that resolves it. Override moves a field to a new owner without any client changing.

Composition

```
COMPOSITION ERROR: Type of field "Order.id" is incompatible across subgraphs:
it has type "ID!" in subgraph "orders" but type "String!" in subgraph "payments"
```

Federation 2 directives

@key	Designates an object type as an entity and specifies its key fields
@shareable	A field is allowed to be resolved by multiple subgraphs
@external	This subgraph usually cannot resolve a field, but needs to define it
@requires	A field resolver depends on entity fields resolved by other subgraphs
@provides	Entity fields a subgraph can resolve, only at a particular path
@override	A field is now resolved by this subgraph instead of another
@inaccessible	Omitted from the router's API schema

Definitions paraphrase Apollo's federation directive reference.

Apollo federation directive reference, checked 2026-09-25

10 Kafka, Spring and Micronaut

Spring names topic and group on the listener and publishes through `KafkaTemplate`. Micronaut puts the group on the class, the topic on the method and the key on a parameter, and publishes through an interface the compiler implements.

orders-spring: `PaymentCapturedListener.java`

```
@Component
public class PaymentCapturedListener {

    private static final Logger log = LoggerFactory.getLogger(PaymentCapturedListener.class);

    private final OrderService orders;
    private final ObjectMapper json;

    public PaymentCapturedListener(OrderService orders, ObjectMapper json) {
        this.orders = orders;
        this.json = json;
    }

    @KafkaListener(topics = "payment-captured", groupId = "orders")
    public void on(String message) {
        PaymentCaptured event = json.readValue(message, PaymentCaptured.class);
        log.info("payment-captured attempt for order {}", event.orderId());
        orders.markPaid(event);
    }
}
```

```
payments-micronaut: OrderPlacedListener.java
```

```
// SYNC commits the offset only after the poll is processed, so a crash before
// that point means the record is delivered again.
// The error strategy is set on purpose: the default, NONE, is deprecated and
// skips every remaining record in the poll when one of them throws.
@KafkaListener(groupId = "payments", offsetReset = OffsetReset.EARLIEST,
    offsetStrategy = OffsetStrategy.SYNC,
    errorStrategy = @ErrorStrategy(value = ErrorStrategyValue.RESUME_AT_NEXT_RECORD))
public class OrderPlacedListener {

    private static final Logger log = LoggerFactory.getLogger(OrderPlacedListener.class);

    private final PaymentService payments;

    public OrderPlacedListener(PaymentService payments) {
        this.payments = payments;
    }

    @Topic("order-placed")
    public void on(@KafkaKey String orderId, OrderPlaced event) {
        log.info("order-placed attempt for order {}", orderId);
        payments.capture(event);
    }
}
```

```
payments-micronaut: PaymentEvents.java
```

```
// No implementation anywhere: Micronaut writes it at compile time.
@KafkaClient
public interface PaymentEvents {

    @Topic("payment-captured")
    void captured(@KafkaKey String orderId, PaymentCaptured event);
}
```

```
What the Micronaut compiler wrote
```

```
$ javap -cp payments-micronaut/target/classes \
    'dev.codewithsam.shop.payments.PaymentEvents$Intercepted'
class dev.codewithsam.shop.payments.PaymentEvents$Intercepted
    implements dev.codewithsam.shop.payments.PaymentEvents, ...
```

The Micronaut build generated `PaymentEvents$Intercepted`; the Spring build generated no comparable class.

Run on a real machine, 2026-09-25. Reproduce it with `scripts/verify.sh` in the course repository.

11 Kafka down at checkout: coupling and the outbox

Publishing inside the request waited for the producer's `max.block.ms`, 60 seconds by default, then failed and rolled the order back. Nothing was lost: a Kafka outage became a checkout outage. The textbook lost event needs a different failure order and was not measured here.

With the outbox, the order and its event commit together; a relay publishes afterwards. Accepted in a fraction of a second with Kafka down, the service killed before publishing, and paid after the restart. The relay can publish twice: at least once, not exactly once.

orders-spring: OrderService.java

```
// The order, its lines and the event commit together, or not at all.
@Transactional
public Order placeOrder(PlaceOrderInput input) {
    long total = input.lines().stream()
        .mapToLong(line → catalog.priceOf(line.sku()) * line.quantity())
        .sum();
    Order order = orders.save(new Order(input.customerId(), input.currency(), total));
    List<OrderLine> saved = input.lines().stream()
        .map(line → new OrderLine(order.getId(), line.sku(), line.quantity(),
            catalog.priceOf(line.sku())))
        .toList();
    lines.saveAll(saved);

    String orderId = order.getId().toString();
    OrderPlaced event = new OrderPlaced(orderId, order.getCustomerId(), total,
order.getCurrency());
    outbox.save(new OutboxEvent("order-placed", orderId, json.writeValueAsString(event)));
    return order;
}
```

orders-spring: OutboxRelay.java

```
@Scheduled(fixedDelay = 500)
@Transactional
public void publishPending() throws Exception {
    for (OutboxEvent event : outbox.findTop100ByPublishedAtIsNullOrderById()) {
        kafka.send(event.getTopic(), event.getMessageKey(), event.getPayload())
            .get(10, TimeUnit.SECONDS);
        event.markPublished();
    }
}
```

In the request: refused after about 60 seconds. Outbox: accepted, killed, then paid.

Run on a real machine, 2026-09-25. Reproduce it with `scripts/verify.sh` in the course repository.

12 At least once, and a message that always fails

A crash after the payment insert and before the offset commit redelivered the order: two payment rows without a unique key, one with it.

Spring's default error handler ran a failing listener 10 times, then logged and moved on. Micronaut's default, NONE, is deprecated and skips every remaining record in the poll: two good orders behind a bad one were never paid. RESUME_AT_NEXT_RECORD paid them; production still needs a plan for the bad record, such as a dead letter or retry topic.

payments-micronaut: Payment.java

```
@MappedEntity
@Indexes(@Index(columns = "order_id", unique = true))
public record Payment(
    @Id @GeneratedValue Long id,
    String orderId,
    long amount,
    String currency,
    PaymentStatus status) {
}
```

payments-micronaut: PaymentService.java

```
@Transactional
public void capture(OrderPlaced event) {
    if (event.total() ≤ 0) {
        throw new IllegalArgumentException("Refusing to capture " + event.total());
    }
    if (payments.existsByOrderId(event.orderId())) {
        return;
    }
    Payment payment = payments.save(new Payment(null, event.orderId(), event.total(),
        event.currency(), PaymentStatus.CAPTURED));
    events.captured(event.orderId(),
```

Default FixedBackOff(0L, 9): ten attempts. "Failures are simply logged after retries are exhausted."

Spring for Apache Kafka reference, error handling, checked 2026-09-25

NONE: "This error strategy will skip over all records from the current offset in the current poll when the consumer encounters an error." Deprecated.

micronaut-kafka 6.1.0 source, ErrorStrategyValue, checked 2026-09-25

13 Limits and authentication

On an unlicensed Apollo Router 2.17, `limits.router.max_depth` refused to start: operation limits need a GraphOS plan. `MaxQueryDepthInstrumentation(5)` in each subgraph refused a depth 7 query, directly and through the router. Depth is not cost: add pagination, cost limits, timeouts, and a persisted operation safelist.

Federation does not solve authentication. The router validates the token and propagates identity in configured headers; graph level rules can live at the router, domain rules in the subgraph that owns them.

orders-spring: GraphQLLimits.java

```
@Configuration
public class GraphQLLimits {

    @Bean
    public MaxQueryDepthInstrumentation maxDepth() {
        return new MaxQueryDepthInstrumentation(5);
    }
}
```

license violation, the router is using features not available for your license: ["Operation depth limiting"]

Run on a real machine, 2026-09-25. Reproduce it with `scripts/verify.sh` in the course repository.

14 Built twice: what actually differed

The GraphQL layer differs most. Kafka consumers look alike with settings in different places; producers do not. In the earlier Spring Boot to Micronaut video, a misspelled repository finder failed the Micronaut build while Spring built it and refused to start.

Spring Boot and Micronaut, job by job

Job	Spring Boot 4.1.1	Micronaut 5.1.5
Resolve a field	@QueryMapping, @SchemaMapping	data fetcher in a factory
Batch a field	@BatchMapping	DataLoader, per request
Resolve an entity	@EntityMapping, idList	Federation.transform, fetchEntities
GraphQL config	spring.graphql.http.path	graphql.enabled, graphql.path
Consume Kafka	@KafkaListener(topics, groupId)	@KafkaListener(groupId) + @Topic
Publish to Kafka	KafkaTemplate	@KafkaClient interface, generated
Kafka config	spring.kafka.*	kafka.bootstrap.servers + annotation
A listener throws	10 attempts, then logged	NONE: deprecated, skips the poll
Wiring	runtime context, AOT optional	compile time, by default



15 The interview questions this answers

What does @key do, and what does the router send to resolve an entity?

Who owns a field, and how do you move one to another service?

Does a schema that composes guarantee clients still work?

Why can federation still cause N plus one, and why a fresh loader per request?

Why does GraphQL return 200 on a failure, and what does nullability change?

When would you choose REST over GraphQL?

Is depth a cost limit, and where should limits live?

Where does authentication happen in a federated graph?

What does an outbox solve, and where is its duplicate window?

What does at least once mean for a consumer that captures money?

What happens by default when a listener throws, in Spring and Micronaut?

What does Micronaut generate at compile time that Spring builds at runtime?

Before a federated graph with Kafka goes near production

1. Every list field that can repeat is batched: batch mapping or a data loader
2. Nullability is chosen per field, knowing how far a null will travel
3. Events are written with the business change, through an outbox
4. Every consumer is idempotent, enforced by the database
5. Every consumer has an explicit error strategy, retry and recovery path
6. Depth, cost and pagination limits exist somewhere, and you know where
7. scripts/verify.sh passes on a fresh clone



Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **X**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- The course repository**
<https://github.com/code-with-sam-dev/spring-micronaut-federation>
- Apollo federation directives**
<https://www.apollographql.com/docs/graphos/schema-design/federated-schemas/reference/directives>
- Spring for Apache Kafka: error handling**
<https://docs.spring.io/spring-kafka/reference/kafka/annotation-error-handling.html>
- Micronaut Kafka**
<https://micronaut-projects.github.io/micronaut-kafka/latest/guide/>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. Apache Kafka is a trademark of the Apache Software Foundation. GraphQL is a trademark of the GraphQL Foundation. Apollo and GraphOS are trademarks of Apollo Graph, Inc. Micronaut is a trademark of the Micronaut Foundation. PostgreSQL is a trademark of the PostgreSQL Community Association. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-25 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.